



“Kazakh-British Technical University” JSC School of
Information Technology and Engineering

Data Collection and Preparation Report

Team: Oracle

Prepared By:

Em Igor.

Smambayev Zhusup

«24» October 2025

Summary

This report examines the relationship between countries' performance at the 2024 Paris Olympics and their economic characteristics - GDP per capita and population size. Medal data was obtained by statically scraping a Wikipedia table, while economic indicators were obtained via the World Bank's open API. Countries were aligned using ISO3 with manual correction of discrepancies in names. After cleaning, a single data frame is formed and derived metrics are calculated, including "medals per million inhabitants," which allows for comparisons between countries of different sizes. Exploratory analysis confirms the expected positive association: large and wealthy economies tend to be at the top of the table, although a significant group of "overperforming" countries significantly exceeds the trend line. The results are visualized using a "GDP per capita $\times$ population" bubble chart (dot size represents medals, color represents region), a horizontal ranking of the top 20 by "medals per million," and a "medals vs. $\log(\text{GDP per capita})$ " regression plot. A choropleth map for the per million metric is additionally provided. The approach is reproducible: the code saves the cleaned dataset and graph images, and selecting the "latest available year" in the World Bank data ensures relevance within reasonable constraints. The limitations include dependence on the Wikipedia page structure, non-simultaneous availability of the latest economic values, and the need to maintain a dictionary of country correspondences; nevertheless, the findings are robust and reveal both a general trend and interesting deviations that can be further explained by non-financial factors in sport.

Data Sources

The primary source of medal data is the public Wikipedia page "2024 Summer Olympics Medal Table," which contains a country-by-country table with the number of gold, silver, and bronze medals won, as well as the overall total. This source is convenient due to its standard HTML table format and the ability to update quickly after competitions. Economic indicators are obtained from the official World Bank API: GDP per capita in current US dollars and total population, as well as a country directory with ISO codes and region. The World Bank data has a clear schema, stable indicator codes, and good country coverage, facilitating programmatic access and subsequent comparison with the medal count. To supplement the raw values, the code calculates derived indicators - population in millions and the number of medals per million inhabitants - to ensure a more accurate comparison for countries of different sizes.

Code

Web Scrapping

Scraping is implemented for a static HTML page without the need for browser emulation. Requests.get with the User-Agent header is used to correctly identify the client and avoid possible server-side rejections. The response markup is parsed using BeautifulSoup. The algorithm for selecting the required table relies not on position, but on the contents of the headers: the code iterates through all `<table class="wikitable">` elements and looks for one where the set of headers includes "Gold," "Silver," and "Bronze." This makes the parser more resilient to changes in the order of tables on the page. Next, cells are extracted from the table rows, discarding string headers and any service strings. Columns are converted to the target set

[Nation, Gold, Silver, Bronze, Total], and numeric fields are converted with `pd.to_numeric(..., errors="coerce")` so that any comment characters or dashes are neatly converted to NaN and can be filtered. Text trailing from country names (e.g., footnotes like [...]) are removed using a regular expression, after which the `medals_total` column is formed as a numeric version of Total. The resulting medals dataframe contains the cleaned and numeric medals of all three denominations and the total value. This content-driven approach and gentle numerical conversion improve the pipeline's robustness to the typical characteristics of Wikipedia pages.

```
WIKI_URL = "https://en.wikipedia.org/wiki/2024_Summer_Olympics_medal_table"

r = requests.get(WIKI_URL, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(r.text, "html.parser")
tables = soup.find_all("table", {"class": "wikitable"})
target = None
for t in tables:
    ths = [th.get_text(strip=True) for th in t.find_all("th")]
    if {"Gold", "Silver", "Bronze"}.issubset(set(ths)):
        target = t
        break
rows = []
for tr in target.find_all("tr"):
    tds = [td.get_text(" ", strip=True) for td in tr.find_all(["td", "th"])]
    if len(tds) < 5:
        continue
    if "Gold" in tds and "Silver" in tds and "Bronze" in tds:
        continue
    if any(x in tds[0].lower() for x in ["rank", "noc", "nation", "team/noc", "team"]):
        continue
    rows.append(tds)
df = pd.DataFrame(rows)
col_candidates = ["Rank", "Nation", "Gold", "Silver", "Bronze", "Total"]
if df.shape[1] >= 6:
    df = df.iloc[:, :6]
    df.columns = col_candidates
for c in ["Gold", "Silver", "Bronze", "Total"]:
    df[c] = pd.to_numeric(df[c], errors="coerce")
df["Nation"] = df["Nation"].str.replace(r"\[.??\]", "", regex=True).str.strip()
df = df.dropna(subset=["Gold", "Silver", "Bronze"])
df["medals_total"] = df["Total"].fillna(0).astype(float)
df = df[["Nation", "Gold", "Silver", "Bronze", "medals_total"]]
medals = df

medals.head(100)
```

API

To obtain economic data, the World Bank's open REST API is used. First, a single request loads the country directory, from which ISO3 codes, official names, and the region name are saved. This is a critical step, because further integration with the medal count occurs through reconciling Wikipedia country names with the official World Bank names. Then, the `get_indicator(iso3_list, indicator, y1=2020, y2=2024)` function is defined. It accepts a list of ISO codes, an indicator code, and a year range, splits the request into portions, sends them to the endpoint, and extracts (country, year, value) pairs from the responses. For each country, the most recent available value in the specified range is taken: the dataframe is sorted by (iso3, year) and the group "tail" is taken to automatically use the latest data. GDP per capita and population are loaded similarly, after which both indicators are merged with the medal dataframe by ISO code. This design—portion-based URL generation and "last available year" aggregation—makes the API component both reliable and time-correct.

```

WB_COUNTRIES = "https://api.worldbank.org/v2/country?per_page=400&format=json"
WB_INDICATOR = "https://api.worldbank.org/v2/country/{codes}/indicator/{ind}?date={y1}:{y2}&format=json&per_page=20000"

data = requests.get(WB_COUNTRIES).json()

items = data[1]
c = pd.DataFrame(items)
c = c.rename(columns={"id": "iso3", "name": "wb_name"})
c["region_name"] = c["region"].apply(lambda x: x.get("value") if isinstance(x, dict) else None)

countries = c[["iso3", "wb_name", "region_name"]]

countries.head()

```

Data Cleaning and Preparation

The key task of data preparation is to reconcile country names from Wikipedia with the official "wb_name" from the World Bank reference book. The code uses the manual_map dictionary for this purpose, which covers typical discrepancies: "Great Britain" -> "United Kingdom," "Czech Republic" -> "Czechia," "Iran" -> "Iran, Islamic Rep.," "Hong Kong, China" -> "Hong Kong SAR, China," "Chinese Taipei" -> "Taiwan, China," along with differences for North Korea/Korean Republic, Congo, Côte d'Ivoire, and so on. This mapping is applied to the Nation column, forming the wb_name, after which the join is performed with the countries table. If any ISO codes remain missing after the merge, the code prints a missing slice so that the dictionary can be supplemented and the step repeated. After successful merging, the calculated features population_million and medals_per_million are formed. These are then used as the basis for normalized comparisons. To quickly assess data quality, describe() is run on key numeric columns and diagnostics about missing GDP per capita values are printed. Finally, the final dataset is saved to the olympics2024_medals_gdp_pop.csv file, making the results reproducible and convenient for reanalysis. Together, these steps ensure correct linking of sources of different natures, explicit handling of exceptions in names, and careful management of missing data.

```

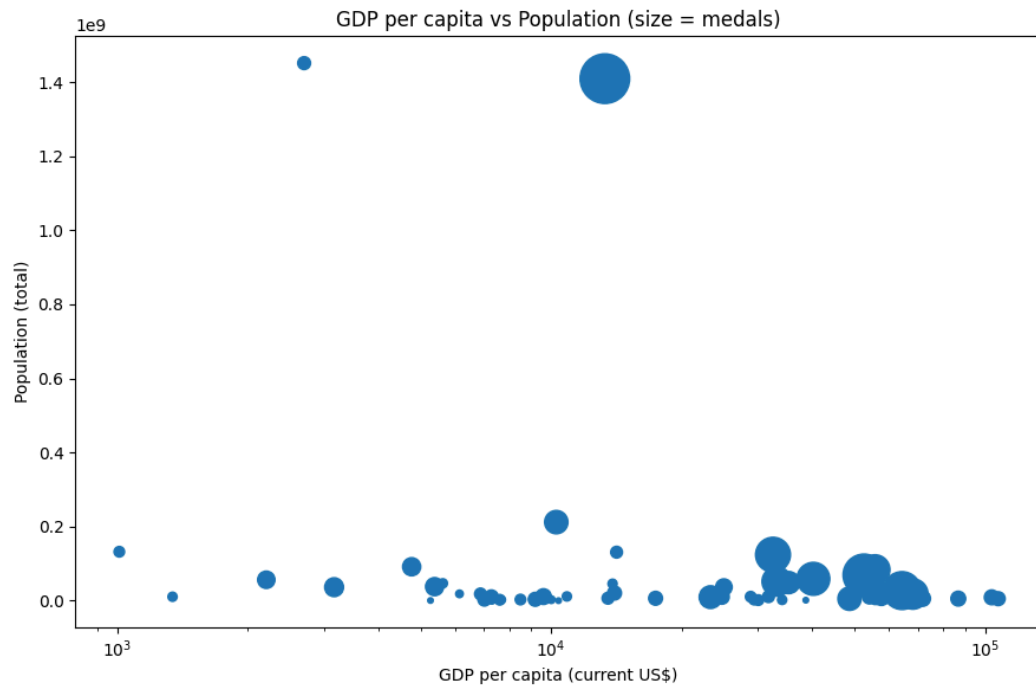
manual_map = {
    "United States": "United States",
    "Great Britain": "United Kingdom",
    "Czech Republic": "Czechia",
    "Iran": "Iran, Islamic Rep.",
    "Hong Kong, China": "Hong Kong SAR, China",
    "Chinese Taipei": "Taiwan, China",
    "South Korea": "Korea, Rep.",
    "North Korea": "Korea, Dem. People's Rep.",
    "Russia": "Russian Federation",
    "Kosovo": "Kosovo",
    "Vietnam": "Vietnam",
    "Syria": "Syrian Arab Republic",
    "Moldova": "Moldova",
    "Ivory Coast": "Cote d'Ivoire",
    "Côte d'Ivoire": "Cote d'Ivoire",
    "DR Congo": "Congo, Dem. Rep.",
    "Congo": "Congo, Rep.",
    "Bolivia": "Bolivia",
    "Venezuela": "Venezuela, RB",
    "Tanzania": "Tanzania",
    "Laos": "Lao PDR",
    "Brunei": "Brunei Darussalam",
    "Cape Verde": "Cabo Verde",
    "Bahamas": "Bahamas, The",
    "Micronesia": "Micronesia, Fed. Sts.",
    "Macao, China": "Macao SAR, China",
    "Trinidad and Tobago": "Trinidad and Tobago",
    "São Tomé and Príncipe": "Sao Tome and Principe",
    "Eswatini": "Eswatini",
    "Timor-Leste": "Timor-Leste"
}

m = medals.copy()
m["wb_name"] = m["Nation"].replace(manual_map)
merged_try = pd.merge(m, countries, left_on="wb_name", right_on="wb_name", how="left")
missing = merged_try[merged_try["iso3"].isna()][["Nation", "wb_name"]].drop_duplicates()
missing.head()

```

Visualisations

GDP per capita vs Population

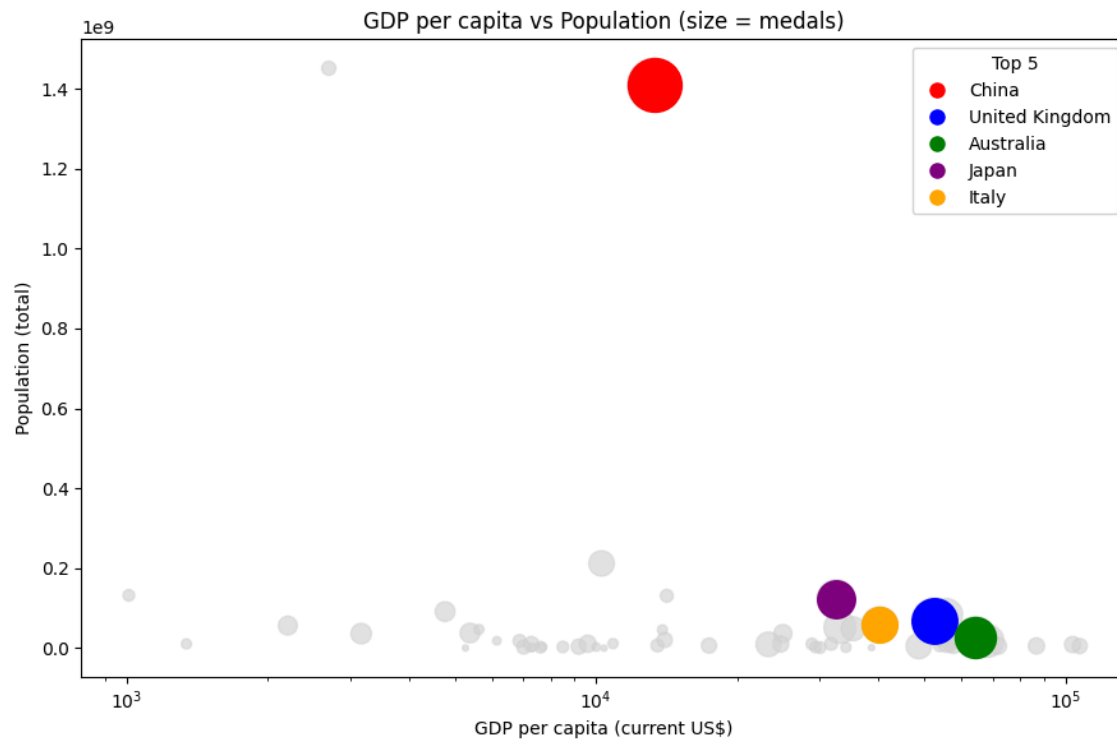


```
plt.figure(figsize=(9,6))
s = (df["medals_total"].fillna(0) + 0.1) * 10
plt.scatter(df["gdp_per_capita"], df["population"], s=s)
plt.xscale("log")
plt.xlabel("GDP per capita (current US$)")
plt.ylabel("Population (total)")
plt.title("GDP per capita vs Population (size = medals)")
plt.tight_layout()
plt.savefig("fig1_bubble_gdp_pop.png", dpi=200)
plt.show()
```

The code plots a bubble chart with GDP per capita on the x-axis, nations on the y-axis, and dot size corresponding to medal ratio. A 9x6 canvas is created; the expression $s = (\text{medals_total}.\text{fillna}(0) + 0.1) * 10$ sets the area of the markers. `plt.scatter(...)` draws the dots, `plt.xscale("log")` compresses the income spread; then connect the axes and title. And we adjust the margins, after which the plot is saved and displayed.

GDP per capita vs Population №2

We added colors to top 5 countries according to medals numbers



```
top_5_medals = df.sort_values("medals_total", ascending=False).head(5)

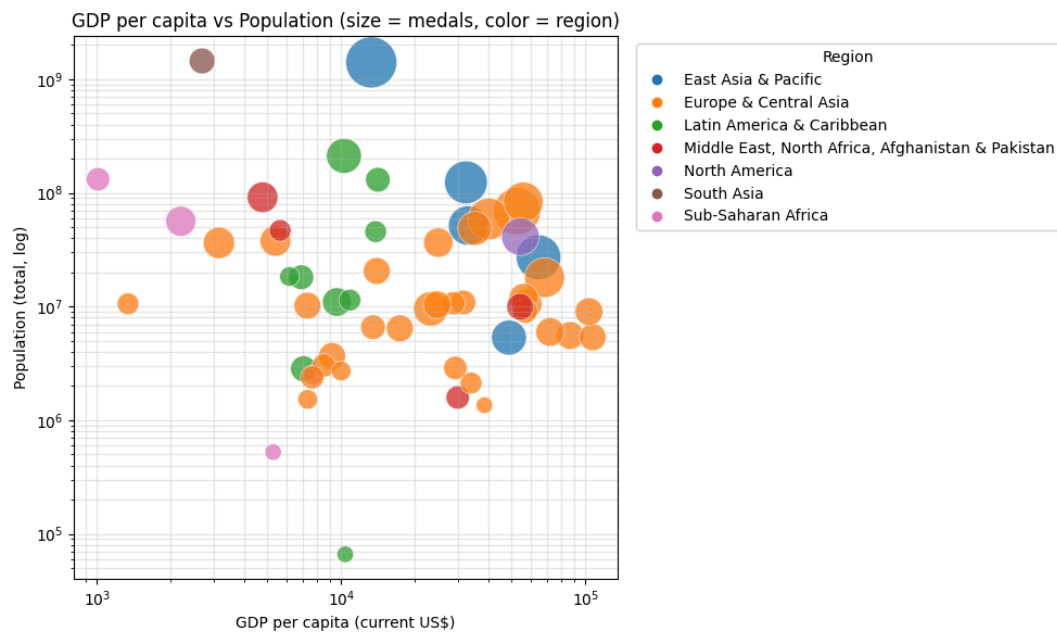
plt.figure(figsize=(9,6))
s = (df["medals_total"].fillna(0) + 0.1) * 10
plt.scatter(df["gdp_per_capita"], df["population"], s=s, color='lightgray', alpha=0.6)

colors = ['red', 'blue', 'green', 'purple', 'orange']
for i, (_, row) in enumerate(top_5_medals.iterrows()):
    plt.scatter(row["gdp_per_capita"], row["population"], s=(row["medals_total"] + 0.1) * 10, color=colors[i], label="_nolegend_")

from matplotlib.lines import Line2D
handles = [
    Line2D([0],[0], marker='o', linestyle='', color=colors[i], label=top_5_medals.iloc[i]["wb_name"], markersize=8) for i in range(len(top_5_medals))
]

plt.xscale("log")
plt.xlabel("GDP per capita (current US$)")
plt.ylabel("Population (total)")
plt.title("GDP per capita vs Population (size = medals)")
plt.legend(handles=handles, title="Top 5")
plt.tight_layout()
plt.savefig("fig1_bubble_gdp_pop_top5.png", dpi=200)
plt.show()
```

GDP per capita vs Population №3



```
df1 = df.dropna(subset=["gdp_per_capita", "population", "medals_total", "region_name"])
regions = df1["region_name"].astype("category")

colors = plt.cm.tab10(regions.cat.codes % 10)

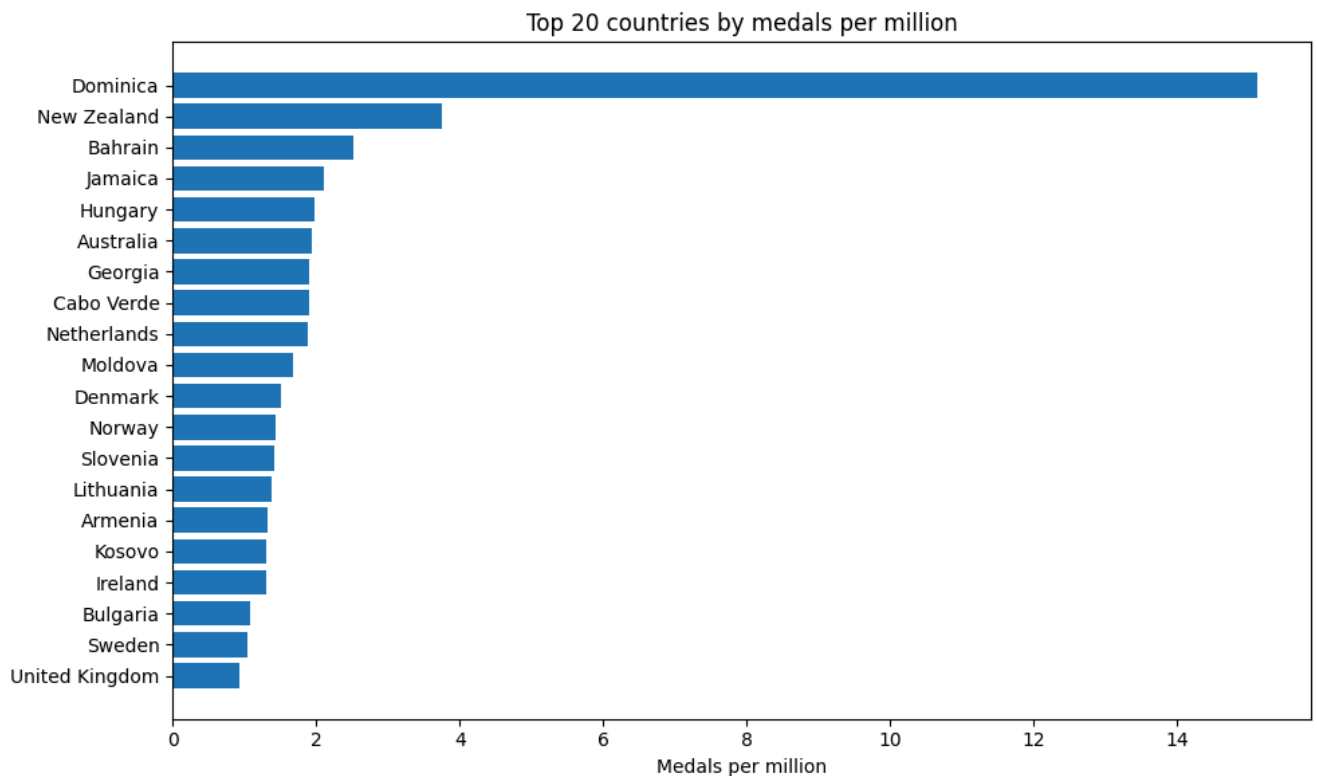
plt.figure(figsize=(10,6))
s = np.sqrt(df1["medals_total"].values) * 120
plt.scatter(
    df1["gdp_per_capita"], df1["population"],
    s=s, c=colors, alpha=0.75, edgecolors="white", linewidths=0.5
)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("GDP per capita (current US$)")
plt.ylabel("Population (total, log)")
plt.title("GDP per capita vs Population (size = medals, color = region)")

cats = list(regions.cat.categories)
handles = [
    plt.Line2D([0],[0], marker='o', linestyle='', color=plt.cm.tab10(i % 10), label=lab)
    for i, lab in enumerate(cats)
]
plt.legend(handles=handles, title="Region", bbox_to_anchor=(1.02,1), loc="upper left")

plt.grid(True, which="both", alpha=0.3)
plt.tight_layout()
plt.savefig("fig1b_bubble_gdp_pop_better.png", dpi=300)
plt.show()
```

Creates a bubble chart: X is GDP per capita, Y is population; both axes are logarithmic. Marker size is $\sqrt{\text{medals_total}} \times 120$, color is region based on the tab10 palette. Gaps are removed; dots are semi-transparent with a white border; regional legends are manually selected and placed outside the chart field; the graph is saved and shown.

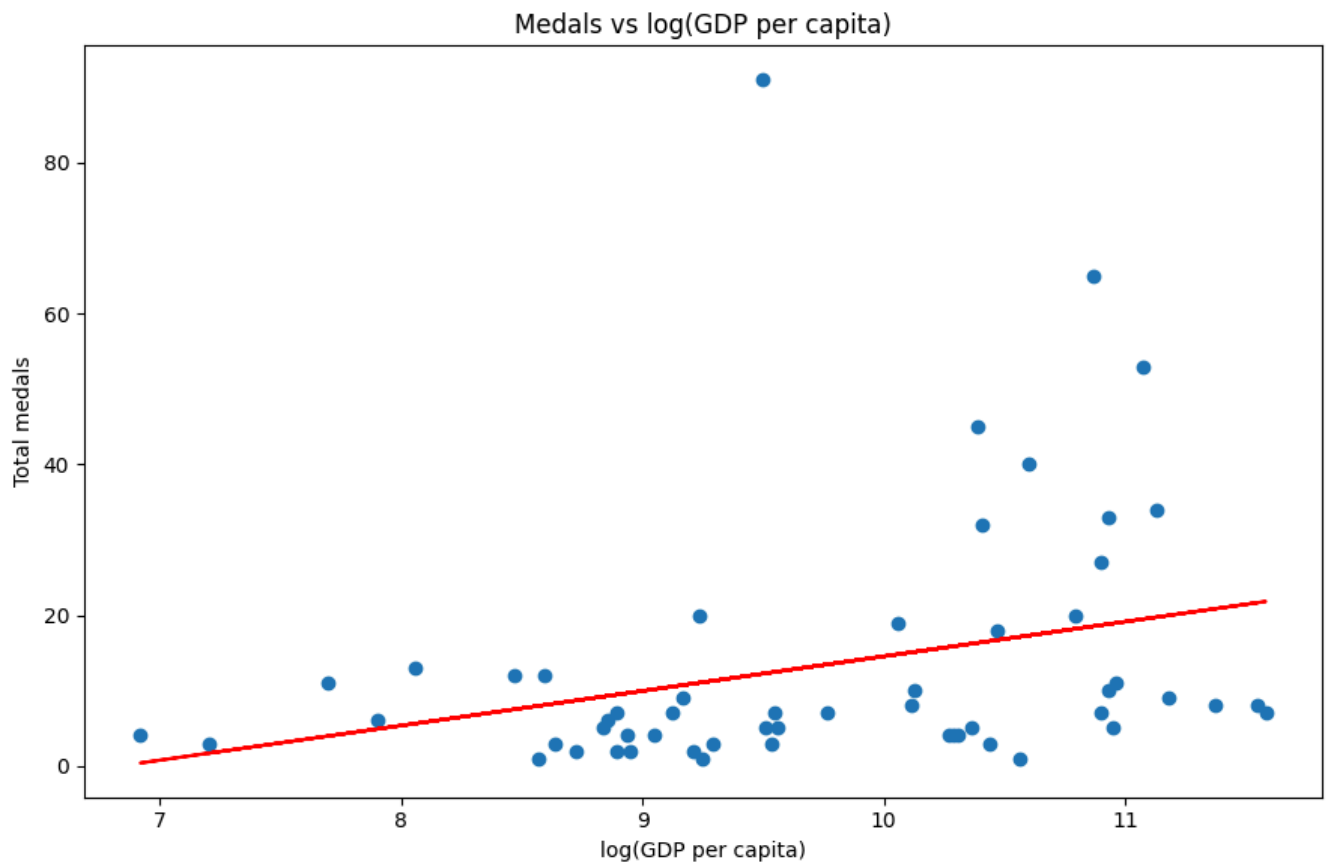
Top 20 countries by medals per million



```
top = df.sort_values("medals_per_million", ascending=False).head(20)
plt.figure(figsize=(10,6))
plt.barh(top["wb_name"], top["medals_per_million"])
plt.gca().invert_yaxis()
plt.xlabel("Medals per million")
plt.title("Top 20 countries by medals per million")
plt.tight_layout()
plt.savefig("fig2_bar_mppm.png", dpi=200)
plt.show()
```

The code generates a country ranking by "medals per million inhabitants" and draws a horizontal bar chart. First, the data frame is sorted by `medals_per_million` in descending order, and the first 20 rows are taken; this is the top selection. Then, a bar chart is plotted on a 10x6 canvas, where the Y axis is the country names, and the X axis is their `medals_per_million` values. The `invert_yaxis()` call reverses the Y axis order so that the leader is on top, the axis labels and title explain the metric and plot, and `tight_layout()` compresses the margins. The chart is then saved as png and displayed.

Medals vs log(GDP per capita)



```
x = np.log(df["gdp_per_capita"].replace(0,np.nan)).dropna()
y = df.loc[x.index, "medals_total"]
coef = np.polyfit(x, y, 1)
yy = coef[0]*x + coef[1]
plt.figure(figsize=(9,6))
plt.scatter(x, y)
plt.plot(x, yy, color='red')
plt.xlabel("log(GDP per capita)")
plt.ylabel("Total medals")
plt.title("Medals vs log(GDP per capita)")
plt.tight_layout()
plt.savefig("fig3_scatter_reg.png", dpi=200)
plt.show()
```

The code plots the "total medals vs. log GDP per capita" scatterplot and applies a linear fit on top. First, the `gdp_per_capita` column is taken, zeros are replaced with NaNs, the logarithm is applied, and then gaps are removed - this creates a vector `x` of only valid positive values. Using the same indices, the target vector `y = medals_total` is selected so that `x` and `y` are strictly aligned across rows. The `np.polyfit(x, y, 1)` function estimates the coefficients of a simple linear regression, after which the trend line $y = a \cdot x + b$ is calculated. Next, a `scatter(x, y)` point cloud is plotted on a new canvas, and a red trendline is plotted on top. The axis labels and title clarify that the horizontal axis represents `log(GDP per capita)` and the vertical axis represents `Total medals`. Calling `tight_layout()` tightens the margins, the graph is saved and displayed.

Conclusion

The results show the expected relationship: the richer and larger the economy, the higher the overall medal count, although some countries significantly exceed the trend. The normalized "medals per million inhabitants" metric and visualizations provide a more fair comparison and help identify anomalies. Limitations relate to the structure of the sources and the asynchronous nature of the years; in future studies, it would be useful to consider GDP at PPP and a proxy for sports investment.